

Received 14 January 2025, accepted 14 March 2025, date of publication 17 March 2025, date of current version 26 March 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3552240

RESEARCH ARTICLE

Enhancing Anti-Money Laundering Frameworks: An Application of Graph Neural Networks in Cryptocurrency Transaction Classification

STEFANO FERRETTI¹, (Member, IEEE), GABRIELE D'ANGELO¹, AND VITTORIO GHINI

Department of Computer Science and Engineering (DISI), University of Bologna, 40126 Bologna, Italy

Corresponding author: Stefano Ferretti (s.ferretti@unibo.it)

This work was supported in part by European Union—NextGenerationEU within the framework of Piano Nazionale di Ripresa e Resilienza (PNRR) Mission 4—Component 2—Investment 1.1 under Italian Ministry of University and Research (MUR) Programme “Progetti di Ricerca d’Interesse Nazionale (PRIN) 2022” through SmartShires—CUP: H53D23003570006 under Grant 2022N2NH42.

ABSTRACT Cryptocurrency money laundering is a pressing issue, as it not only facilitates and hides criminal activities but also disrupts markets and the overall financial system. To respond this challenge, researchers are trying to develop robust Anti-Money Laundering (AML) frameworks. These efforts play a crucial role in promoting societal welfare by mitigating the impact of criminal activities. This paper explores the application of Graph Neural Networks (GNNs) for classifying Bitcoin transactions. The research specifically employs Graph Convolutional Networks (GCNs), Graph Attention Networks (GATs), the Chebyshev spatial convolutional neural networks, and GraphSAGE networks. Based on the dataset analysis, we experiment with different subsets of features. Our findings suggest that the use of Graph Neural Network convolutions, combined with a final linear layer and skip connections, allow for an improvement in the state-of-the-art results, especially when Chebyshev and GATv2 convolutions are used.

INDEX TERMS Anti-money laundering, deep learning, graph neural networks, classification.

I. INTRODUCTION

Cryptocurrency money laundering stands out as a significant challenge today. Anonymity and decentralized exchange represent attractive features to exploit and obfuscate the flow of money that have been raised through illegal activities. Criminals initiate the process by converting fiat currency into cryptocurrencies. These digital assets are then shuffled through a complex web of transactions, mixing services, and exchanges. The goal is to obscure the money trail, making it challenging to trace the flow of funds [1].

In recent years, several solutions have been investigated on how to combat, or at least mitigate, this increasing problem. Examples are legal frameworks establishing “obliged entities” with Anti-Money Laundering (AML) and Countering the Financing of Terrorism (CFT) responsibilities, as well

as the attempt to harmonize AML efforts through the development of international standards and directives [2], [3].

Another approach to combat money laundering relates to the use of machine learning techniques to classify transactions. Previous works already demonstrated that traditional machine learning classification techniques, such as Decision Trees and Random Forests, are able to provide good classification results [4], [5]. These first solutions suggest that further research can be done, that uses more sophisticated approaches based on the specific characteristics of cryptocurrencies platforms.

Many cryptocurrency platforms are based on the Unspent Transaction Output (UTXO) model. In this model, a transaction has an input and output. When a transaction is made, a user takes one or more UTXOs to serve as the inputs that are used to transfer (and thus spend) those cryptos the user earned in these inputs. The outputs of this transaction become novel UTXOs that can be used later, in novel transactions. This movement of assets can be represented as a (directed,

The associate editor coordinating the review of this manuscript and approving it for publication was Bing Li¹.

acyclic) transaction graph that can be analyzed [6]. This observation suggests that Graph Neural Networks (GNNs) might be viable classification models to exploit in this context [3]. In fact, GNNs have shown promise in capturing the relational information present in graph-structured data, making them suitable for analyzing transaction networks in cryptocurrencies.

This work follows this general idea. We consider different types of GNNs, i.e., Graph Convolutional Networks (GCNs), Graph Attention Networks (GATs), Chebyshev convolutional networks, and GraphSAGE networks, using different variations of architectural models. For each GNN type, we explore various versions to validate if they enhance transaction classification. Specifically, we experimented with i) a baseline model that consists of two graph convolutional layers; ii) a variant that extends the previous architecture by adding a final linear layer, with the aim to refine the learned representations and improve classification performance; iii) an architecture with a final linear layer and skip connections, which allow information to flow directly from early layers to later ones, enhancing feature propagation and potentially boosting accuracy.

Moreover, we also conducted a comprehensive analysis of the Elliptic dataset, used in this work. This analysis helps to understand the characteristics of the dataset and to identify its limitations. In fact, the dataset suffers from a class imbalance, which can impact model performance. We also dive into the features used for classification. We notice that certain features exhibit strong correlations with each other. This correlation may affect the interpretability of the model and may introduce redundancy. Thus, in our analysis, we explore whether the removal of highly correlated features improves classification accuracy.

Our findings show that the use of aggregated features improves the final results in general. In particular, selecting a subset of aggregated features and reducing the number of correlated ones provide a marginal increase in the final performance, with respect to the use of the whole feature set. As concerns the GNN models, Chebyshev convolutions, followed by the GATv2 variant, surpass other techniques and advance the current state of the art in terms of recall and F1 scores. The results demonstrate the efficacy of various classification methods and underscore the potential of GNNs to improve the detection and prevention of AML. It is also worth noticing that adding a final linear layer and skip connections improve the models' performance.

On the other hand, the opacity of the dataset under examination complicates the explainability of the machine learning system and fails to provide insights for comprehensively understanding the forensic strategies to combat money laundering phenomena. In substance, our analysis underscores the necessity of utilizing multiple and novel datasets, including synthetically generated ones, to study and understand money laundering mechanisms on cryptocurrency platforms.

To summarize, the main contributions of this work are the following ones.

- We conduct an in-depth study on the Elliptic dataset, which provides insights into feature selection;
- We demonstrate that the use of Chebyshev and GATv2 GNNs enhance classification performance;
- We show that incorporating skip connections and a final linear layer in GNN models can lead to substantial performance improvements. Skip connections, by creating additional pathways for information flow in the network, help mitigate the problem of vanishing gradients and lead to more robust learning;
- We highlight the critical need of interpretable and usable datasets that can provide deeper insights into how and when to classify transactions.

The remainder of this paper is structured as follows. Section II provides a background on AML and GNNs. Section III describes a detailed analysis of the dataset. Section IV discusses the methodology for the classification study, thus describing the employed classifiers and the considered metrics. Section V is focused on the results of the classification using different GNN classifiers. Finally, Section VI provides some concluding remarks.

II. BACKGROUND

In this section, we delve into the background related to AML within the realm of cryptocurrencies and then, we overview the main methods we employ for the classification of transactions.

A. ANTI-MONEY LAUNDERING IN CRYPTOCURRENCIES

AML is a critical concern in the digital currency landscape. Money laundering represents an illegal activity where ill-gotten gains are masked through legitimate businesses, camouflaging their illicit origins. The primary objective is to elude law enforcement agencies and avoid traceability. With the advent of cryptocurrencies, criminals have discovered a novel way to carry out this illegal activity [7]. Indeed, cryptocurrencies, with their decentralized nature and pseudonymous transactions, present unique challenges for AML efforts. Here are some key issues:

- **Anonymity and Pseudonymity:** Unlike traditional financial systems, where identities are tied to accounts, cryptocurrencies allow users to transact pseudonymously. This anonymity complicates efforts to trace illicit activities.
- **Global and Borderless Transactions:** Cryptocurrencies operate across borders without geographical constraints. Tracking suspicious transactions becomes complex when funds move seamlessly between jurisdictions.
- **Complex Transaction Networks:** The intricate graph of transactions involving multiple addresses, exchanges, and wallets makes it challenging to identify patterns associated with money laundering. Moreover, other approaches such as mixing services exist to keep users

crypto-anonymity in the transparent ledger of Bitcoin network, which are being vastly exploited by criminals for laundering illegal money [8].

- **Emergence of Anonymity-Enhanced Cryptocurrencies:** Anonymity Enhanced Cryptocurrencies, also called Privacy-focused cryptocurrencies, (such as Monero and Zcash) enhance user privacy by obfuscating transaction details. These coins pose additional hurdles for AML investigators.

The absence of robust regulation poses a challenge for authorities attempting to trace and unmask money launderers operating within the cryptocurrency realm. This concern passes across governments, financial institutions, and regulatory bodies worldwide. Consequently, tailored AML frameworks have emerged, aiming to grapple with the unique intricacies of digital currencies [9].

Various strategies have been proposed and implemented to combat money laundering. Among these, the most prominent technique is the enforcement of Know Your Customer (KYC) and AML policies by cryptocurrency exchanges and other service providers. These policies necessitate users to submit identification documents and other personal details to authenticate their identity and inhibit money laundering activities [3]. Consequently, users are required to disclose personal information such as their name, address, and date of birth. By making it challenging for criminals to open accounts under false identities, KYC serves as a potent deterrent to money laundering.

An additional technique involves the employment of blockchain analysis tools. These tools are capable of tracing the flow of funds across the blockchain and pinpointing suspicious transactions. They leverage a variety of algorithms and heuristics to discern patterns in transaction data, which could potentially signal money laundering activities. The following are some examples of possible techniques.

- **Heuristics and Rule-Based Approaches:** These involve predefined rules and thresholds to flag suspicious transactions. For instance, sudden large transfers, frequent transactions, or connections to known illicit addresses trigger alerts [10].
- **Address Clustering:** Grouping related addresses (belonging to the same user or entity) aids in tracking funds. Clusters can reveal money flows and connections.
- **Behavioral Analysis:** Monitoring user behavior over time helps detect deviations from normal patterns. Unusual transaction frequencies, amounts, or destinations raise red flags.
- **Machine Learning Models:** ML algorithms analyze transaction data to identify patterns indicative of money laundering [11]. These models learn from historical data and adapt to evolving tactics used by criminals. For instance, in [4] the authors compare several classification techniques, showing how Random Forest classifiers can provide interesting performance results.

- **Graph Analysis and Graph ML:** By visualizing transaction networks as graphs, investigators can uncover hidden relationships. Graph-based techniques reveal clusters, hubs, and anomalous behavior [12], [13]. Transaction graphs can be used as the dataset for ML techniques. For example, GCNs have been used in some recent works, providing interesting results [3], [7], [14], [15]. In this work, we continue that path by experimenting further with Graph Neural Networks (GNNs) classifiers.

B. TRANSACTION FRAUD DETECTION

The problem of identifying illicit transactions in a (distributed) ledger surely resembles that of identifying frauds in more traditional economic systems. In fact, transaction fraud has been a persistent challenge in financial systems, evolving dramatically with the rise of digital payment technologies [16]. Historically, fraud detection relied on manual review and rule-based systems, which were inherently limited in their ability to adapt to sophisticated fraud techniques. The exponential growth of electronic transactions has necessitated more advanced, data-driven approaches to detect and prevent fraudulent activities. Thus, modern fraud detection techniques have shifted towards more dynamic and intelligent approaches that leverage machine learning algorithms [17], [18], [19], [20], [21], [22].

For example, in [23] a model based on siamese neural networks is proposed to identify online fraud transactions. In [24], convolutional neural networks are instead used for similar purposes. In [25], an AdaBoost transfer is used for the classification. In [26], a distributed deep learning framework is presented for the detection of credit card fraud.

Similarly to what we propose in this paper, some other works use graph analysis and graph-based machine learning for fraud identification. For instance, in [27] a GNN-based fraud detection model is presented that leverages sub-graph generators and neighborhood samplers to handle category imbalance. The specific model used is a classic GNN, while in this work we focus on different types of graph-based neural network models. The contribution of this work is, however, interesting since, as we will see in the rest of the paper, the problem of imbalanced datasets is particularly evident in this context. Thus, some strategies are needed to cope with this limitation that affects the classification of transactions.

In [28], a model is proposed named CAMouflage-REsistant GNN (CARE-GNN). The approach uses label-aware similarity, together with reinforcement learning and graph-based aggregation techniques, to perform fraud detection. Although this work is not related to AML or cryptocurrencies, it provides interesting insights on the use of graph-based ML techniques.

It is also worth mentioning that there are works that present datasets in the domain of financial fraud. For example, DGraph is a real-world dynamic graph in the finance domain [29]. It contains about 3M nodes, 4M dynamic edges, and 1M ground-truth nodes. The interesting insight from this

work is that it reveals that anomalous nodes and normal nodes generally have different structures, neighbor distribution, and temporal dynamics. This suggests that the use of graph-based ML approaches can be beneficial in the classification of fraud detection in general.

C. GRAPH NEURAL NETWORKS

Graph Neural Networks (GNNs) are designed to process graph-structured data. They can be broadly divided into two categories, i.e. spectral and spatial methods. Typically, GNNs calculate node representations through an iterative procedure. Each node u has an associated representation embedding $\mathbf{h}_u^{(l)}$, which is updated in every iteration l . These iterations can be thought of as layers in a conventional neural network.

1) SPECTRAL METHODS

These methods use a convolutional operator in the spectral domain [30]. The primary steps in the spectral methods for GNNs are:

- 1) Graph Laplacian Construction: Given a graph, its Laplacian matrix is computed as $L = D - A$, where D is the degree matrix, i.e., a diagonal matrix where each element in the diagonal D_{ii} represents the degree of vertex i , and A is the conventional adjacency matrix.
- 2) Spectral Transformation: The features of the graph nodes are transformed into the spectral domain using the eigenvectors of the graph Laplacian matrix.
- 3) Filtering: Spectral filters are applied. Filtering operations are analogous to convolutions in conventional image-based convolutional neural networks.
- 4) Inverse Spectral Transformation: The processed spectral representations are transformed back into the original domain using the inverse eigenvectors transformation.

Chebyshev convolution and Graph Convolutional Networks belong to this type of methods.

2) CHEBYSHEV CONVOLUTIONAL NEURAL NETWORK

Chebyshev spectral convolutional neural network approximates the spectral filter using Chebyshev polynomials, reducing the computational complexity [30]. In particular, a Chebyshev expansion of order K is used to deal with the problem of locality. In this way, only a K -hop neighborhood is considered for the computation of the convolutional operations.

Spectral graph convolutions are usually described through the convolution of a signal $\mathbf{x}$ with a filter $\mathbf{g}_\theta$ [31]. In this case, the filter takes the form

$$\mathbf{x} *_G \mathbf{g}_\theta = \sum_{i=0}^{K-1} \theta_i \mathbf{T}_i(\tilde{\mathbf{L}}) \mathbf{x}$$

where K is the order of the Chebyshev polynomial approximation, θ_i are the learnable Chebyshev coefficients (weights), $\mathbf{T}_k(\tilde{\mathbf{L}})$ are the degree- k Chebyshev polynomials of the first kind and $\tilde{\mathbf{L}}$ the normalized Laplacian using the largest

eigenvalue $\lambda_{\max}(\mathbf{L})$ of L , i.e.,

$$\tilde{\mathbf{L}} = \frac{2L}{\lambda_{\max}(\mathbf{L})} - \mathbf{I}_n$$

being $\mathbf{I}_n$ the $n \times n$ identity matrix. The idea is that the computation starts with $\mathbf{h}^{(0)} = \mathbf{x}$, and then filters are iteratively applied, together with a non linear function, to obtain features for nodes in the graph.

The update of the features in a Chebyshev convolution for node u at layer $l + 1$ can be represented as follows [31]:

$$\mathbf{h}_u^{(l+1)} = \sigma \left(\sum_{k=0}^{K-1} w_k \mathbf{T}_k(\tilde{\mathbf{L}}) \mathbf{h}_u^{(l)} \right)$$

where $\mathbf{h}_u^{(l+1)}$ represents the input features of the graph at layer $l + 1$, $\sigma()$ is a non-linear activation function, K is the order of the Chebyshev polynomial approximation, w_k are the learnable Chebyshev coefficients (weights), $\mathbf{T}_k(\tilde{\mathbf{L}})$ are the degree- k Chebyshev polynomials of the first kind and $\tilde{\mathbf{L}}$ the already mentioned normalized Laplacian.

3) GRAPH CONVOLUTIONAL NETWORKS

GCNs simplify the spectral approach by limiting the approximation to first-order neighborhoods ($K = 1$) [32]. GCNs have since become one of the most popular methods for graph-based learning tasks due to their simplicity and efficiency. Of course, using only a 1-hop neighborhood, as the GCN does, leads to a smaller capacity to learn some generalization. Thus, GCN can be considered as aggregating feature information from a node's neighborhood. The features updates for a node u in layer $l + 1$ can be represented as

$$\mathbf{h}_u^{(l+1)} = \sigma \left(W \sum_{v \in \mathcal{N}(u)} \frac{\mathbf{h}_v^{(l)}}{|\mathcal{N}(u)|} + B \mathbf{h}_u^{(l)} \right)$$

where $\mathbf{h}_u^{(l+1)}$ represents the updated feature representation of node u in layer $l + 1$, $\mathcal{N}(u)$ denotes the set of neighboring nodes of node u , W and B represent the weight parameters to be learned, $\mathbf{h}_v^{(l)}$ is the feature representation of node v in layer l , and $\sigma()$ is a non-linear activation function.

4) SPATIAL METHODS

Spatial methods perform convolutions directly on the graph. This involves an initial transformation of node features, followed by their update based on the information of the node and its neighbors [33].

Inspired by the concept of message passing, spatial methods aggregate node features within a local neighborhood. The fundamental principle is to iteratively update a node's representation by aggregating information from its neighbors [34]. This process entails a series of message passing steps where each node exchanges information with its neighbors and updates its feature representation accordingly. This mechanism allows nodes to capture both local and global information, thereby enabling the GNN to effectively learn complex graph structures.

The primary steps in spatial methods for GNNs are as follows:

- 1) Initialization: Each node is initialized with an initial feature vector, typically representing its raw input attributes or embeddings.
- 2) Message Passing and Aggregation: Nodes communicate with their neighbors by exchanging their current feature representations. These messages are then aggregated within each node's local neighborhood to capture pertinent information from neighboring nodes.
- 3) Update Function: An update function is applied to each node to integrate the aggregated information and update its feature representation.
- 4) Iterations: The message passing, aggregation, and update steps are repeated for multiple iterations, enabling nodes to refine their representations by incorporating information from distant parts of the graph.

5) GRAPHSAGE

GraphSAGE (Graph Sample and Aggregated) is a seminal spatial method introduced in [34]. GraphSAGE operates in an inductive learning setting, which means that it can be generalized to unseen nodes during training. GraphSAGE operates by sampling a fixed-size set of neighboring nodes and then aggregating their feature representations using a differentiable aggregator function (e.g., mean, max, LSTM). This process allows GraphSAGE to capture the local graph structure efficiently and create meaningful node embeddings that encode both local and global information. The update of the features in GraphSAGE for node u can be represented as follows:

$$\mathbf{h}_u^{(l+1)} = \sigma \left(W \left[\mathbf{h}_u^{(l)} || \text{AGG}_{v \in \mathcal{N}(u)} \left(\left\{ \mathbf{h}_v^{(l)} \right\} \right) \right] \right)$$

where: $\mathbf{h}_u^{(l+1)}$ represents the updated feature representation of node u in layer $l + 1$, W represents the weight parameters to be learned, $||$ is the concatenation operator, $\mathcal{N}(u)$ denotes the set of neighboring nodes of node u , and $\text{AGG}()$ is a function that aggregates the feature representations of the neighboring nodes.

6) GRAPH ATTENTION NETWORKS

Another spatial method is GAT (Graph Attention Network) [35]. GAT introduces the concept of attention mechanisms into graph learning, enabling the model to learn node representations by assigning different attention weights to neighboring nodes during aggregation. This attention mechanism substitutes the statically normalized convolution operation of GCNs. In substance, the attention mechanism allows the model to focus more on important neighbors while ignoring irrelevant ones, resulting in more informative and context-aware node embeddings. By using attention, GAT can capture complex relationships and dependencies within the graph, making it particularly effective in tasks where certain neighbors are more influential than others.

A scoring function learns the weight matrix W , i.e. a shared parameter for all nodes in the graph, and the attention weights a , i.e. attention scores between a node and its neighbors. These attention scores determine how much importance each neighbor should receive during the aggregation process. The update of the features in GAT for node u can be represented as follows:

$$\mathbf{h}_u^{(l+1)} = \sigma \left(\sum_{v \in \mathcal{N}(u) \cup u} \alpha_{uv}^{(l)} \mathbf{h}_v^{(l)} \right)$$

where: $\mathbf{h}_u^{(l+1)}$ represents, as before the updated feature representation of node u in layer $l + 1$, W represents the weight parameters to be learned, $\mathcal{N}(u)$ denotes the set of neighboring nodes of node u , and $\alpha_{uv}^{(l)}$ are attention weights generated by an attention mechanism, that measure the connective strength between the node u and v .

We used GAT networks in two variants, which are directly implemented using PyTorch Geometric built-in layers. In the standard GAT, the aggregation step is as follows [35]:

GAT:

$$\begin{aligned} \alpha_{uv}^{(l)} &= \log_softmax \left(e_{uv}^{(l)} \right) \\ e_{uv}^{(l)} &= g \left(a^T [W h_u^{(l)} || W h_v^{(l)}] \right) \end{aligned}$$

where $e_{uv}^{(l)}$ are usually referred as attention scores, $g()$ is a *LeakyReLU()* activation function, a is a vector of learnable parameters, as well as usual weights W [36]. The $\log_softmax()$ function ensures that the attention weights sum up to one over all neighbors of the node u [33]. In this original version, the weights W and a are applied consecutively, and can be collapsed into a single linear layer.

The GATv2 modifies the order of operations. Thus, the a layer is applied after the non-linearity and the W layer after the concatenation. It has been proved that this dynamic attention provides a much better robustness to noise [37].

GATv2:

$$e_{uv}^{(l)} = a^T g \left(W [h_u^{(l)} || h_v^{(l)}] \right).$$

III. ANALYSIS OF THE DATASET

In our study, we utilized the Elliptic transactions dataset [4]. This dataset comprises actual Bitcoin transactions structured as a directed graph network. Each transaction is represented as a node, and the directed edges between them illustrate the flow of funds from the source to the destination address. The dataset encompasses a total of 203,769 transaction nodes, interconnected by 234,355 edges. Each transaction node is characterized by 167 features. The initial 94 features are directly related to the transaction itself, extracted from the blockchain, and include details such as the number of inputs and outputs of a transaction. The remaining 73 features are associated with the graph network and are derived from transactions neighboring a specific node. It is important to note that due to intellectual property concerns, the features are not accompanied by descriptions [4].

TABLE 1. Statistics on the number of nodes and their degree.

Type	# Nodes	Degree Stats					
		Mean	Std	Min	25%	50%	Max
All	203769	2.30	4.33	1.00	1.00	2.00	473.00
Licit	42019	3.10	7.78	1.00	1.00	2.00	473.00
Illicit	4545	2.01	7.18	1.00	1.00	1.00	177.00

TABLE 2. Comparison of network metrics for different classes.

Metrics	All	Licit	Illicit
Degree	1.13×10^{-5}	1.52×10^{-5}	9.87×10^{-6}
Centrality	$\pm 2.12 \times 10^{-5}$	$\pm 3.82 \times 10^{-5}$	$\pm 3.52 \times 10^{-5}$
Clustering	0.014	0.008	0.0004
Coefficient	± 0.097	± 0.073	± 0.016
Closeness	0.0021	0.0024	0.0018
Centrality	± 0.0012	± 0.0013	± 0.0010

Each transaction in the dataset is categorized as “illicit”, “licit”, or “unknown”. Table 1 shows the statistics on the number of nodes and their degree for different types of transactions, i.e., all the transactions, licit and illicit ones. For each class, the table reports the number of nodes, as well as some statistics related to their degree, i.e., the mean, standard deviation, minimum, 25th percentile, 50th percentile, and maximum values of the degree distribution for each type of network. Some observations are worthy. First, it is interesting to observe that mean degree of licit transactions (3.10), is higher than illicit ones (2.01) and all transactions in general. The transaction with highest degree is a licit one (473), and in general labeled transactions have a higher standard deviation with respect to the one measured for the whole network. This is probably due to the fact that the amount of unlabeled transactions is significantly higher than labeled ones. However, it seems that the illicit transaction set has the lowest variability in the degree distribution, as indicated by the slightly smaller standard deviation (7.18) with respect to the licit subset, the narrow range between the minimum and maximum values (1-177), and the fact that the 50th percentile (or median) of the degree distribution is 2 for the whole network and the licit class, while it is equal to 1 for the illicit class. This means that half of the nodes in the global net and the licit subnet have 2 or fewer connections, while half of the nodes in the illicit subset have only 1 connection.

Table 2 compares some centrality metrics for the three different classes of transactions in the network, i.e., all the transactions, licit, and illicit ones. The degree centrality measures how well-connected a node is to other nodes in the network [38]. The higher the degree centrality, the more connections a node has. The table shows that licit nodes have the highest degree centrality, with respect to all nodes, and illicit nodes in particular.

The clustering coefficient measures how clustered a node is with its neighbors. A high clustering coefficient means that a node’s neighbors are also likely to be neighbors of each other [39]. The table shows that in general, the statistics collected for all the nodes have the highest

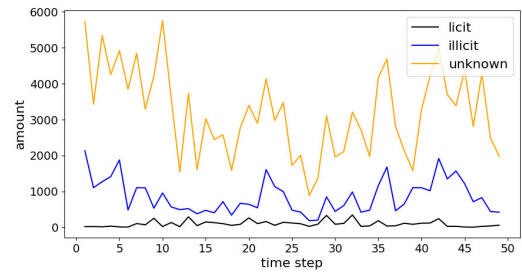


FIGURE 1. Amount of transactions per time step for each category (i.e., label).

clustering coefficient, followed by licit nodes, and then illicit nodes. Thus, in general the unlabeled transactions are more clustered, while illicit transaction have a lower tendency to cluster with other transactions. This is indeed an outcome that should be expected.

The closeness centrality measures how close a node is to all other nodes in the network [38]. The shorter the distance, the closer the node is to all other nodes. The table shows that licit nodes have the highest closeness centrality, with respect to the whole network and to the illicit nodes. Overall, the table shows that licit nodes tend to be more central and well-connected than illicit nodes.

Figure 1 depicts the distribution of transactions across time steps, categorized by their labels. It is important to note that the dataset has anonymized timestamps, providing only a relative ordering without revealing actual timing or Bitcoin blockchain block association. Despite this limitation, visual analysis suggests potential correlations between unknown and illicit transactions, as evidenced by peaks occurring at the same time steps. Additionally, licit transactions appear to exhibit some peak overlap with both illicit and, to a lesser extent, unknown categories.

The original dataset description does not explain the features in detail due to intellectual property concerns. We only know that some features are related to the single transaction under consideration, while others are derived from aggregating values from neighbouring transactions in the transaction graph. The inputs for the aggregated features are also not provided. To better understand the aggregated features, we plot their correlation with the local features in Figure 2. More specifically, on the x-axis, we provide the timestep and local features, while on the y-axis the aggregated features are reported. We observe that the correlations vary widely. Most of the aggregated features have weak or insignificant correlations with the local features. Only a few aggregated features have a strong association with a subset of local features. Another interesting finding is that some of the aggregated features have very similar patterns of correlation with the local features.

Based on the observation of strong correlations between certain aggregated features, Figure 3 delves deeper into the relationships within this group. It reveals several feature pairs exhibiting correlations exceeding 0.8, often occurring

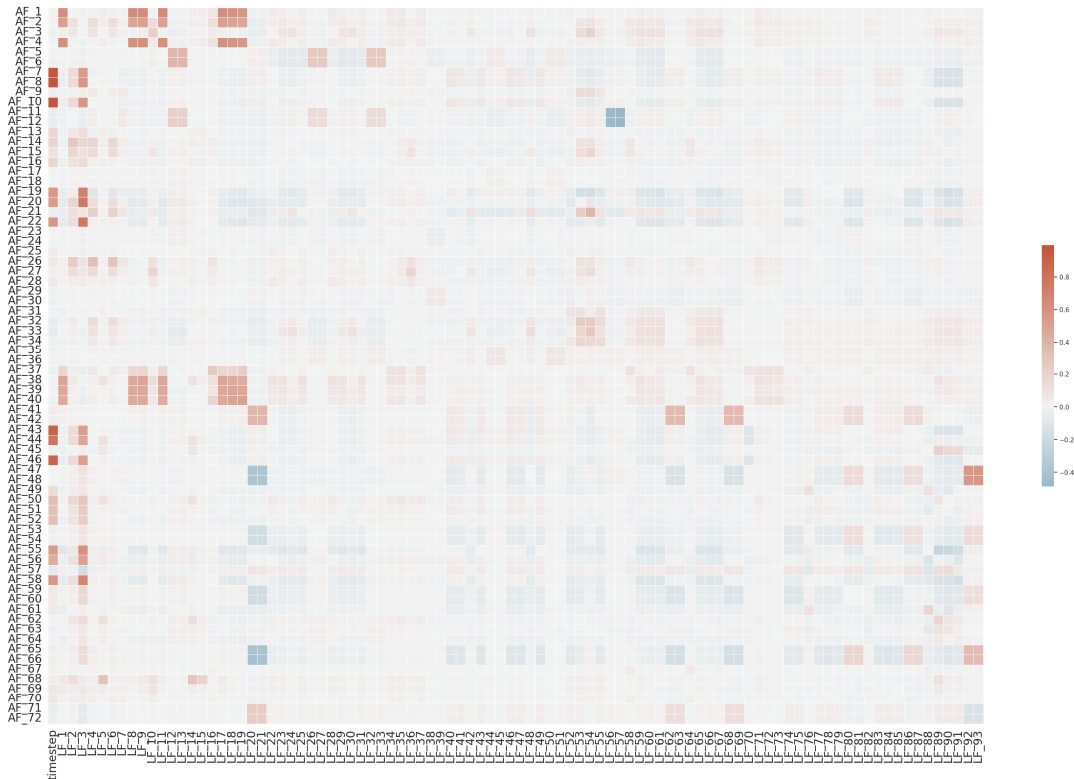


FIGURE 2. Correlation between Local Features (LF) and Aggregated Features (AF).

consecutively within the data. These significant correlations raise the important question if all features are equally necessary for effective classification.

Figure 4 visualizes scatter plots of aggregated features exhibiting exceptionally strong correlations, exceeding 0.9. These high-correlation pairs further demonstrate of a clear and direct relationship between these aggregated features.

IV. METHODOLOGY

In this section, we discuss the models we compare in this analysis study. We will outline the performance metrics, as well as the details related to the specific GNNs employed and the parameters set during the training.

A. THE PERFORMANCE METRICS

The performance of the models was evaluated using the following metrics: precision, recall, and F1 score, specifically for the illicit class, along with the micro-averaged F1 score considering all classes.

- **Precision:** This metric quantifies the ratio of true positive predictions to all positive predictions made by the model. A high precision score signifies that the model generates fewer false positives, thereby accurately identifying illicit transactions when it categorizes a transaction as illicit.
- **Recall:** This metric concentrates on false negative errors, providing an indication of missed positive

predictions, i.e., illicit transactions that were incorrectly classified as licit. Essentially, it assesses the model's ability to correctly identify all illicit transactions.

- **F1 score:** This metric seeks a balance between precision and recall by computing their harmonic mean. It thus offers a balanced measure of a model's performance.
- **Micro-averaged F1 score:** This version of the F1 score calculates the metric at the micro-level, taking into account the total number of true positives, false positives, and false negatives across all classes. In essence, it represents the proportion of correctly classified observations out of all observations, encompassing both licit and illicit transactions.

It is important to notice that we computed Precision, Recall and F1 score for the licit class as well, but we will omit these results for the sake of conciseness. As expected, the classification of the licit transactions is much simpler, thus providing better results for this particular class, with respect to the illicit one.

In view of the considerations of Section III, we consider different sets of features, with the aim to assess the impact of the transactions, besides the performance of the different classifiers. Thus, we employ only features related to the transactions ("tx" configuration), the whole set of features available in the original dataset ("tx+agg") and a reduced set of aggregated features together with the features associated to the specific transactions ("tx+agg_red"). In particular,

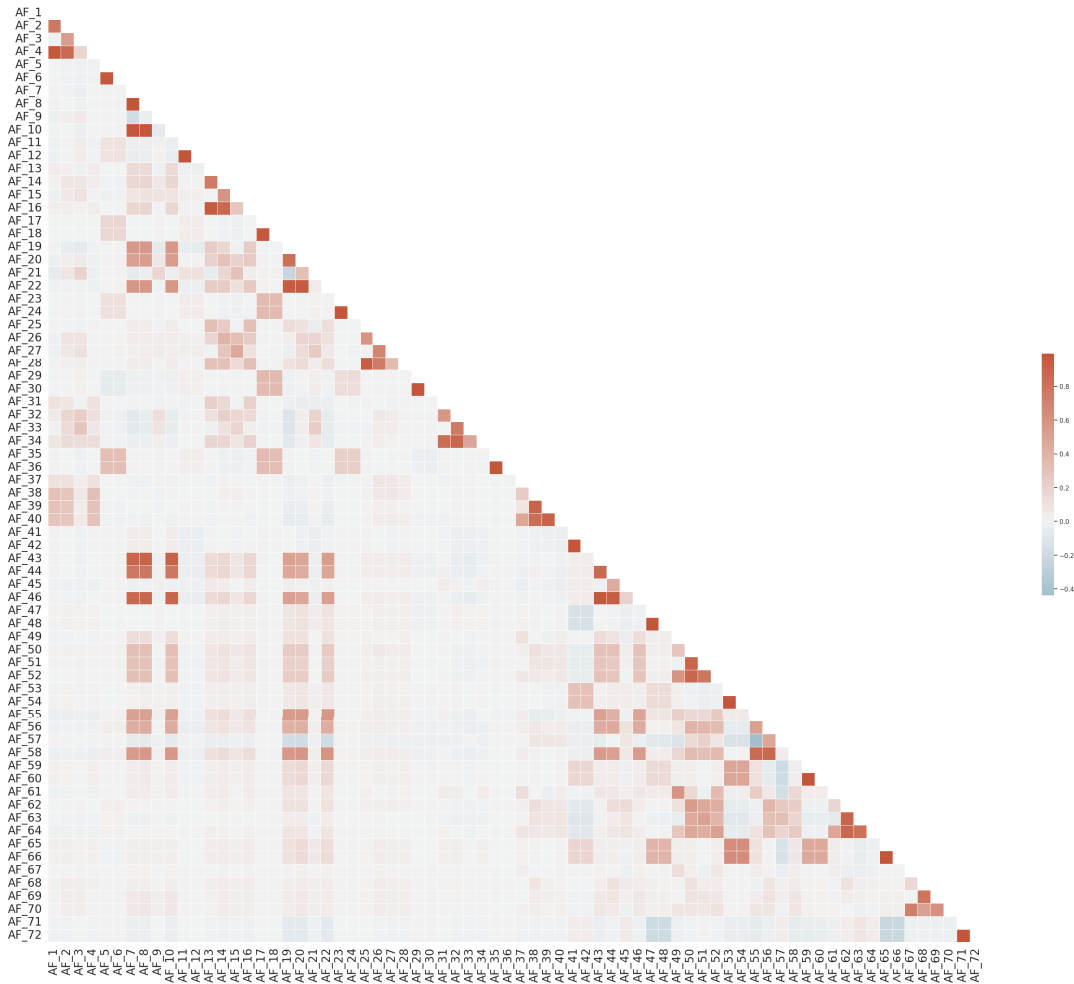


FIGURE 3. Correlation between aggregated features.

in this set we excluded all the aggregated features that showed an extraordinary correlation with some other feature. In the results of the experiments reported in this paper, the correlation threshold used to exclude features was kept equal to 0.9, that allowed removing 22 features in total.

B. THE EMPLOYED CLASSIFIERS

During the training of the employed GNNs, the following hyper-parameters for all the models were used:

- **Binary Cross Entropy** as a loss function;
- **Adam optimizer** with a dynamic learning rate starting from $9e-3$ and reduced by a factor of 0.9 if validation loss does not improve for 100 epochs;
- **Hidden Units** were set equal to 64 when only features of the transactions were used (tx configuration), while it was set to 110 in the configuration with all features (tx+agg) and the reduced set of aggregated features (tx+agg_red);
- **13000 epochs** with a early stopping callback with a patience of 1000 epochs.

We employed three variants for each GNN model, as depicted in Figure 5. Depending on the specific convolution layer used within the GNN, we obtained the GNNs listed in Table 3. The specific design implementations described in Section II are incorporated within the convolution layers.

1) GRAPH NEURAL NETWORK WITH TWO STACKED CONVOLUTIONAL LAYERS

The first type of network architecture was a sequence of two graph convolutional layers.

Layers Description:

- 1) **First Convolution Layer:** It applies a graph convolution operation on the input graph. It uses ReLU activation function. Dropout is applied to prevent overfitting during training.
- 2) **Second Graph Convolution Layer:** It is similar to the first layer, but it takes as input the output of the first layer. The output of this layer is the final output of the network.

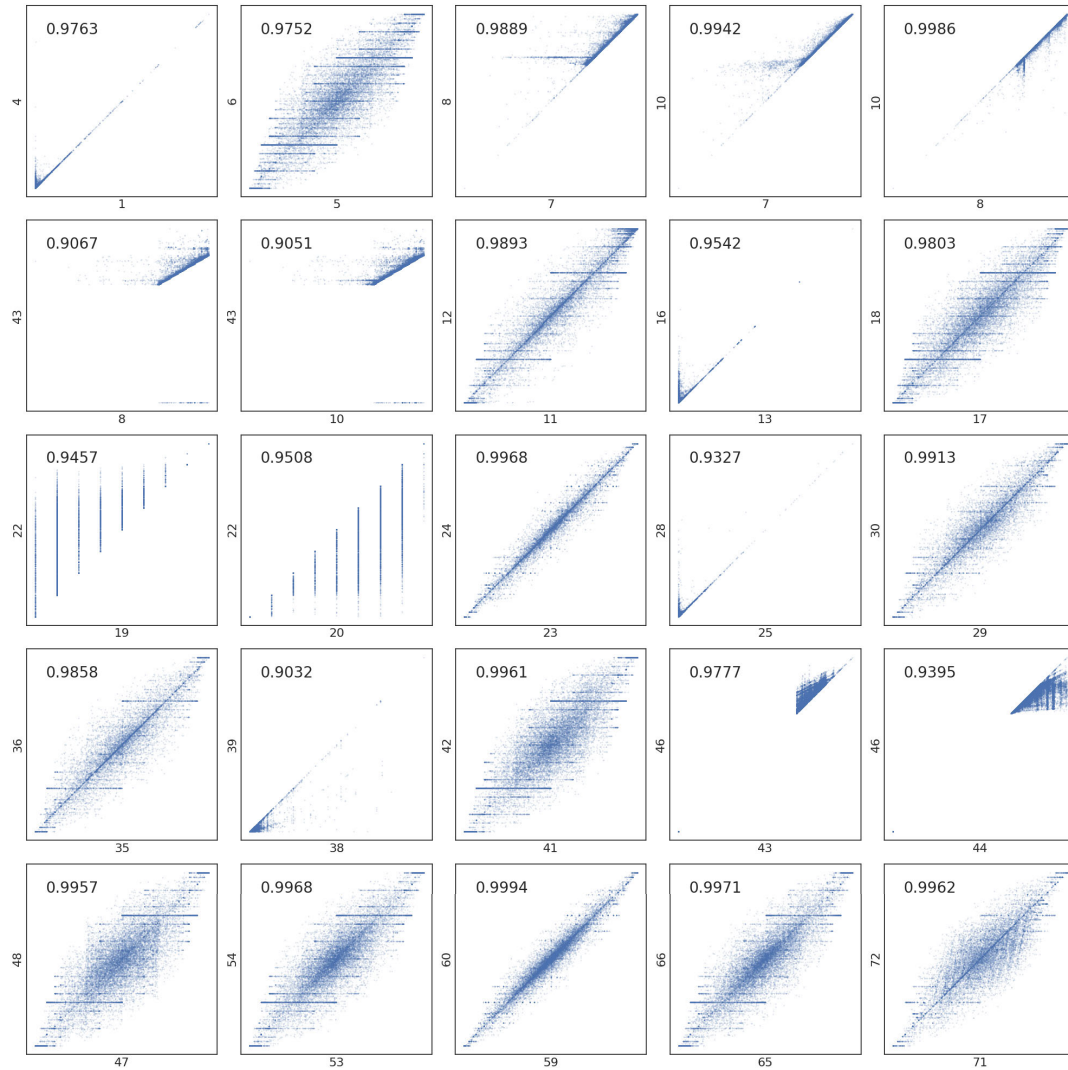


FIGURE 4. Pairs of aggregated features with correlation higher than 0.9. Correlation values shown inside each graph.

TABLE 3. GNNs implemented based on the type of convolutional layer.

Type of Convolutional Layer	Name of GNN
GraphConv	GCN
SAGEConv	SAGE
GATConv	GAT
GATv2Conv	GATv2
ChebConv	Chebyshev

2) GRAPH NEURAL NETWORK WITH A FINAL LINEAR LAYER

This model is the GNN described before, with the addition of a final Linear layer.

Layers Description:

- 1) **First Convolution Layer:** It applies a graph convolution operation on the input graph. It uses ReLU activation function and applies dropout during training.
- 2) **Second Graph Convolution Layer:** Similar to the first layer but it takes as input the output of the first

layer. Also this layer uses ReLU activation function and applies dropout during training. The output of this convolution layer produces an embedding of the same size of the first convolution.

- 3) **Linear Layer:** It applies a linear transformation to the output of the second layer. The output of this layer is passed through a log softmax function to obtain the final output probabilities for each class.

The obtained GNNs, based on the specific graph convolution used, is referred hereinafter with the name of the GNN followed by a “Lin” label, e.g., “GCN+Lin”.

3) GRAPH NEURAL NETWORK WITH A FINAL LINEAR LAYER AND SKIP CONNECTIONS

A recognized problem of GNN is that message passing techniques may occur in over-smoothing, i.e. information aggregated from the node neighbors during message passing begins to dominate the updated node representations.

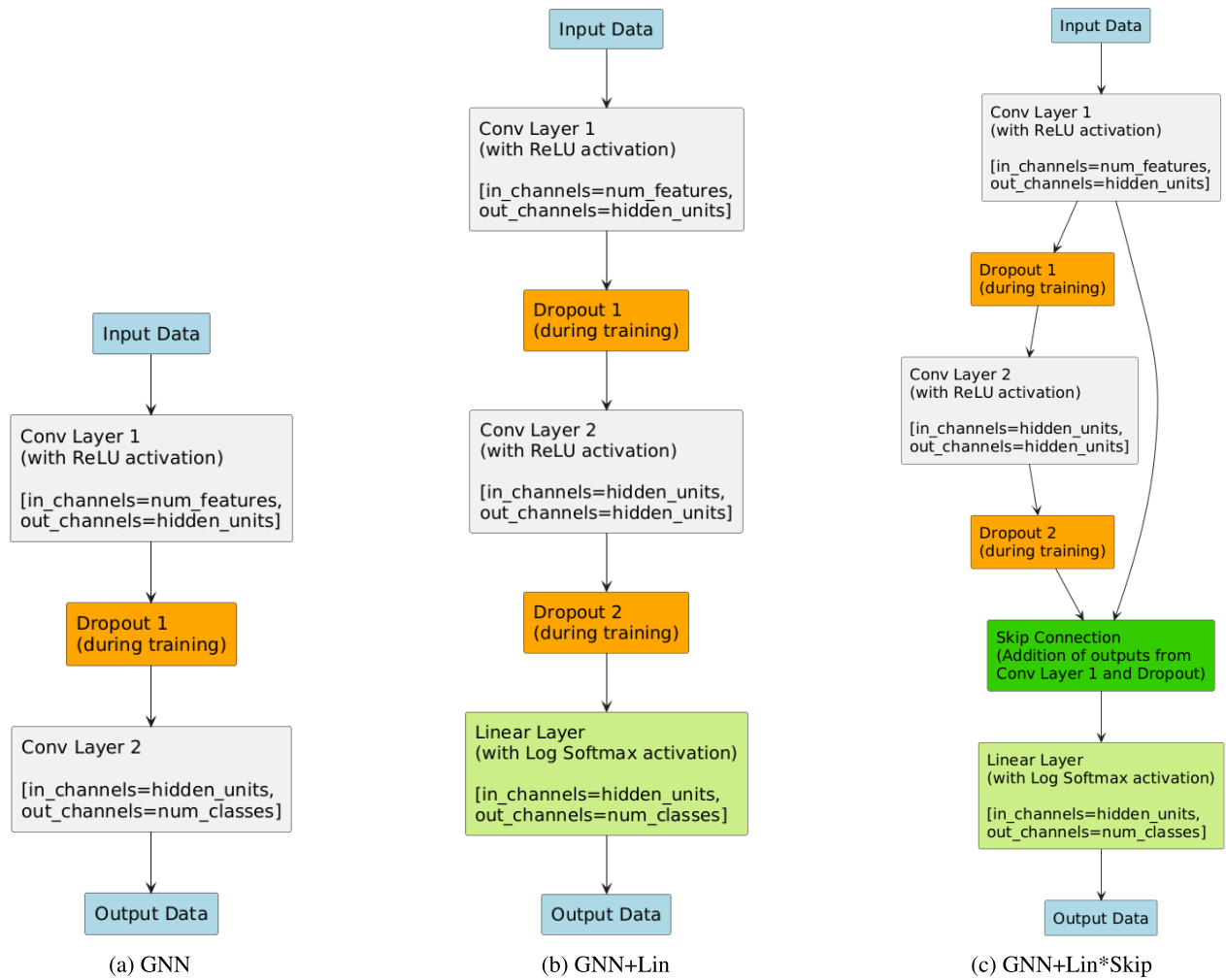


FIGURE 5. The three different variants of GNNs. The specific GNN depends on the specific type of Graph Convolutional Layer.

A natural way to alleviate this issue is to use skip connections, which preserve information from previous rounds of message passing during the update steps [40]. This variant of GNN incorporates the additional linear layer mentioned above and skip connections.

Layers Description:

- 1) **First Graph Convolution Layer:** It applies a graph convolution operation on the input graph. It uses a ReLU activation function and applies dropout during training.
- 2) **Second Graph Convolution Layer:** It is similar to the first layer but it takes as input the output of the first layer. It also uses a ReLU activation function and applies dropout during training. The output of this convolution layer produces an embedding of the same size of the first convolution.
- 3) **Skip Connection:** It adds the output of the first layer to the output of the second layer.
- 4) **Linear Layer:** It applies a linear transformation to the output of the skip connection. The output of this layer

is passed through a log softmax function to obtain the final output probabilities for each class.

The obtained GNNs, based on the specific graph convolution used, is referred hereinafter with the name of the GNN followed by a “Lin+Skip” label, e.g., “GCN+Lin+Skip”.

C. EXPERIMENTAL SETUP AND PARAMETERS

The tests were conducted on a High-Performance Computing (HPC) cluster provided by the Department of Computer Science and Engineering at the University of Bologna. The cluster consists of processing nodes, each equipped with a single octa-core CPU and 64 GB of RAM. Each node contains an Nvidia L40 graphics card (GPU Ada Lovelace AD102GL with 18,176 cores and 48 GB of memory), driven by Nvidia v. 535 drivers and CUDA 11.8 computation libraries. Only a single HPC node was used during the tests.

For each scenario considered, we repeated the tests 15 times, presenting the average results along with the standard deviation.

TABLE 4. Performance of different Graph Neural Network models. For each metric, mean values are reported (with standard deviation).

Model and Configuration	Precision	Recall	F1	M.A. F1
Chebyshev (tx)	0.955 (0.006)	0.789 (0.016)	0.864 (0.009)	0.976 (0.002)
Chebyshev (tx+agg)	0.957 (0.009)	0.852 (0.009)	0.901 (0.007)	0.982 (0.001)
Chebyshev (tx+agg_red)	0.957 (0.012)	0.851 (0.015)	0.901 (0.006)	0.982 (0.001)
Chebyshev+Lin (tx)	0.958 (0.011)	0.790 (0.018)	0.866 (0.010)	0.976 (0.002)
Chebyshev+Lin (tx+agg)	0.956 (0.012)	0.861 (0.013)	0.905 (0.009)	0.983 (0.002)
Chebyshev+Lin (tx+agg_red)	0.955 (0.015)	0.863 (0.012)	0.906 (0.008)	0.983 (0.001)
Chebyshev+Lin+Skip (tx)	0.943 (0.019)	0.812 (0.026)	0.872 (0.012)	0.977 (0.002)
Chebyshev+Lin+Skip (tx+agg)	0.940 (0.014)	0.861 (0.020)	0.898 (0.010)	0.981 (0.002)
Chebyshev+Lin+Skip (tx+agg_red)	0.943 (0.015)	0.859 (0.018)	0.899 (0.008)	0.981 (0.001)
GAT (tx)	0.839 (0.015)	0.659 (0.016)	0.738 (0.012)	0.954 (0.003)
GAT (tx+agg)	0.844 (0.018)	0.727 (0.016)	0.781 (0.010)	0.960 (0.002)
GAT (tx+agg_red)	0.842 (0.021)	0.728 (0.017)	0.780 (0.010)	0.960 (0.002)
GAT+Lin (tx)	0.873 (0.017)	0.648 (0.020)	0.744 (0.015)	0.957 (0.003)
GAT+Lin (tx+agg)	0.843 (0.015)	0.715 (0.027)	0.774 (0.015)	0.959 (0.002)
GAT+Lin (tx+agg_red)	0.840 (0.021)	0.723 (0.024)	0.777 (0.014)	0.960 (0.002)
GAT+Lin+Skip (tx)	0.875 (0.018)	0.735 (0.028)	0.799 (0.013)	0.964 (0.002)
GAT+Lin+Skip (tx+agg)	0.878 (0.027)	0.786 (0.020)	0.829 (0.010)	0.969 (0.002)
GAT+Lin+Skip (tx+agg_red)	0.881 (0.041)	0.774 (0.043)	0.823 (0.032)	0.968 (0.006)
GATv2 (tx)	0.938 (0.009)	0.779 (0.012)	0.851 (0.006)	0.973 (0.001)
GATv2 (tx+agg)	0.934 (0.013)	0.842 (0.011)	0.886 (0.010)	0.979 (0.002)
GATv2 (tx+agg_red)	0.941 (0.012)	0.844 (0.015)	0.890 (0.008)	0.980 (0.001)
GATv2+Lin (tx)	0.944 (0.018)	0.790 (0.019)	0.860 (0.010)	0.975 (0.002)
GATv2+Lin (tx+agg)	0.937 (0.015)	0.846 (0.014)	0.889 (0.011)	0.980 (0.002)
GATv2+Lin (tx+agg_red)	0.943 (0.012)	0.844 (0.017)	0.890 (0.011)	0.980 (0.002)
GATv2+Lin+Skip (tx)	0.933 (0.014)	0.810 (0.017)	0.867 (0.008)	0.976 (0.001)
GATv2+Lin+Skip (tx+agg)	0.926 (0.029)	0.850 (0.020)	0.886 (0.011)	0.979 (0.002)
GATv2+Lin+Skip (tx+agg_red)	0.932 (0.026)	0.848 (0.022)	0.888 (0.022)	0.979 (0.004)
GCN (tx)	0.828 (0.024)	0.453 (0.012)	0.586 (0.014)	0.938 (0.003)
GCN (tx+agg)	0.800 (0.016)	0.496 (0.009)	0.612 (0.009)	0.939 (0.002)
GCN (tx+agg_red)	0.801 (0.013)	0.493 (0.015)	0.610 (0.013)	0.939 (0.002)
GCN+Lin (tx)	0.889 (0.079)	0.457 (0.019)	0.602 (0.015)	0.941 (0.004)
GCN+Lin (tx+agg)	0.876 (0.017)	0.503 (0.013)	0.639 (0.009)	0.945 (0.002)
GCN+Lin (tx+agg_red)	0.880 (0.023)	0.494 (0.017)	0.632 (0.015)	0.944 (0.002)
GCN+Lin+Skip (tx)	0.825 (0.030)	0.522 (0.013)	0.639 (0.012)	0.943 (0.003)
GCN+Lin+Skip (tx+agg)	0.758 (0.011)	0.596 (0.012)	0.667 (0.008)	0.942 (0.002)
GCN+Lin+Skip (tx+agg_red)	0.769 (0.021)	0.584 (0.024)	0.663 (0.019)	0.942 (0.003)
SAGE (tx)	0.939 (0.011)	0.774 (0.021)	0.848 (0.010)	0.973 (0.002)
SAGE (tx+agg)	0.935 (0.013)	0.818 (0.010)	0.873 (0.008)	0.977 (0.002)
SAGE (tx+agg_red)	0.939 (0.008)	0.814 (0.015)	0.872 (0.008)	0.977 (0.001)
SAGE+Lin (tx)	0.944 (0.013)	0.772 (0.017)	0.849 (0.010)	0.973 (0.002)
SAGE+Lin (tx+agg)	0.929 (0.014)	0.808 (0.015)	0.864 (0.009)	0.975 (0.002)
SAGE+Lin (tx+agg_red)	0.935 (0.011)	0.803 (0.014)	0.864 (0.009)	0.975 (0.001)
SAGE+Lin+Skip (tx)	0.892 (0.061)	0.795 (0.025)	0.839 (0.023)	0.970 (0.006)
SAGE+Lin+Skip (tx+agg)	0.918 (0.015)	0.818 (0.015)	0.865 (0.010)	0.975 (0.002)
SAGE+Lin+Skip (tx+agg_red)	0.913 (0.027)	0.821 (0.016)	0.864 (0.009)	0.975 (0.002)

V. RESULTS

In this section, we will present the results obtained in terms of performance and accuracy of the machine learning models, followed by an analysis of computation times and a final discussion.

A. MODELS PERFORMANCE

Table 4 shows the results we obtained with the different models. As already mentioned, precision, recall, and F1 score specifically relate to the illicit class, while the micro-averaged F1 score considers both licit and illicit classes. For each model, we consider different dataset features variants, i.e., “tx” with transaction features only, “tx+agg” with all transactions, and “tx+agg_red” with a subset of (non-excessively correlated) aggregated features.

In the table, for each metrics we show the mean value obtained from multiple repetitions of the same analysis, and the standard deviation (values between parentheses).

It seems that the models that use both transaction features and their neighbor transactions (“tx+agg”) generally perform better than those using only transaction features (“tx”). This suggests that incorporating information from neighboring transactions can improve the model’s performance. As concerns the features set that takes only a reduced set of aggregated features (“tx+agg_red”), in general it does not add significantly noticeable improvements, but nevertheless this solution allows obtaining the highest performance.

The addition of linear layers (Lin) and skip connections (Skip) generally improves model performance, especially evident in the GAT architecture where such additions lead to substantial improvements in recall and F1 scores. However,

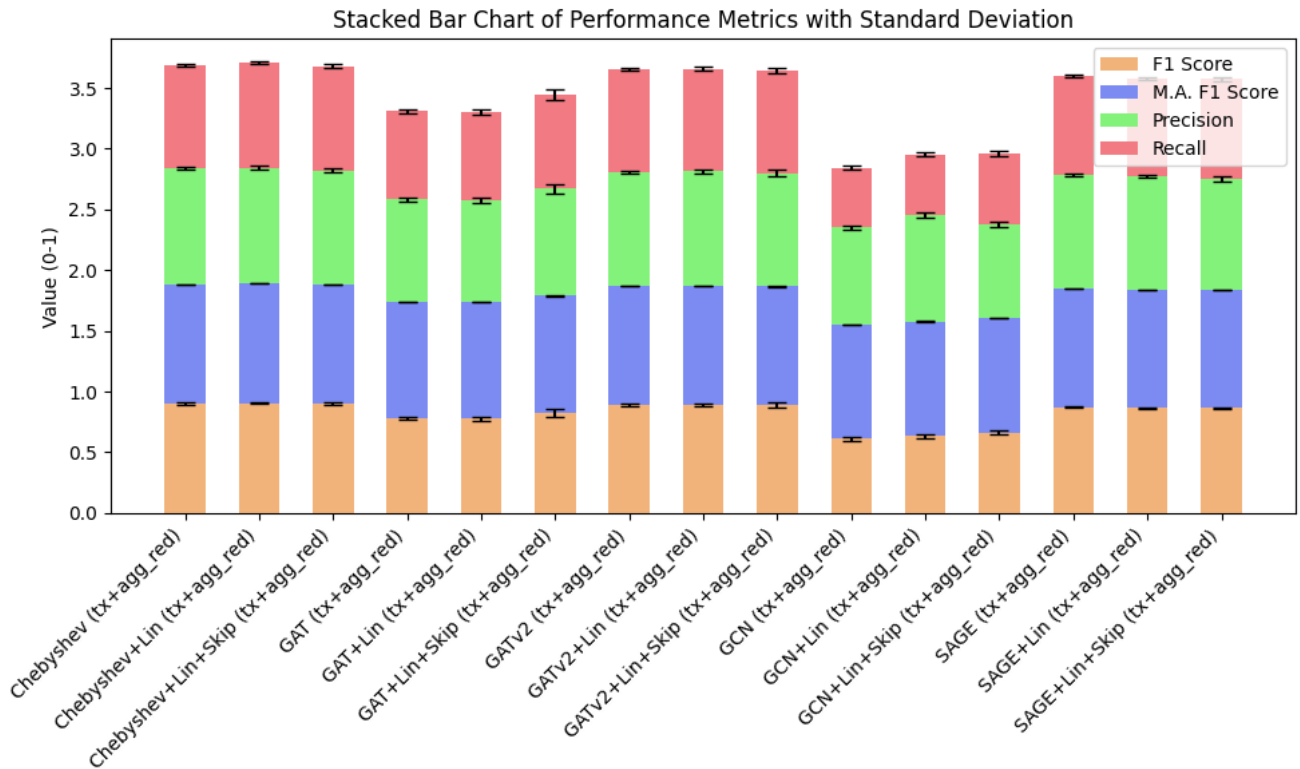


FIGURE 6. Stacked final metrics by classifier for illicit transactions, when using “tx+agg_red” features. For each metrics, mean values are shown with their standard deviation.

this pattern is not uniform across all architectures, indicating that the effectiveness of these modifications depends on the base architecture.

Chebyshev-based models achieve higher performance, with the Chebyshev+Lin (“tx+agg_red”) variant reaching the highest F1 score of 0.906 and Macro-Average F1 (M.A. F1) of 0.983. This configuration also maintains a well-balanced precision-recall trade-off (0.955 and 0.863, respectively). GATv2 models also show robust performance, particularly in configurations that incorporate aggregated features, achieving F1 scores above 0.885. The basic SAGE architectures demonstrate competitive results, with the (“tx+agg”) variant achieving the highest recall (0.818) among all tested models. In contrast, GCN-based models exhibit relatively lower performance across all metrics, with F1 scores not exceeding 0.667, suggesting limitations in capturing the underlying graph structure for this specific task.

It should be noted that these results outperform other studies using graph convolutional networks [3], [7], [14], [15]. As concerns other types of classifiers, in a previous work [3], classic machine learning techniques were used in the attempt to classify illicit transactions in the same dataset and with a similar configuration. Among the considered classification approaches, the best approach was a Random Forest classifier, that was able to reach a F1 score equal to

0.782 and a recall of 0.651, well below the results obtained in this work.

In view of these outcomes, Figure 6 shows a stacked version of the same results, but focusing on the “tx + agg_red” dataset only, which should provide an overall idea of the models that perform better. The figure shows how the Chebyshev convolutions performed better than the others, followed by GATv2.

It is worth mentioning that we explored the impact of feature removal on the performance of our graph-based model. Specifically, we conducted tests where we completely removed the features provided in the Elliptic dataset, leaving only the graph structure and adding metrics of the nodes of the graph itself. However, these modifications led to very poor results. This outcome underscores the critical role that features, particularly those obtained through investigative forensics techniques, play in the model’s performance. It suggests that the graph structure alone, even with added node metrics, is insufficient for our model to effectively capture the necessary information. This result further highlights the importance of robust feature engineering and extraction, obtained through investigative forensics techniques.

B. COMPUTATIONAL TIMES

Table 5 presents the training times (in seconds) for various machine learning models, including their mean and standard

TABLE 5. Mean and standard deviation of the training time for various machine learning models and configurations.

Model and Configuration	Training Time (mean $\pm$ std sec)
Chebyshev (tx)	45.95 (1.03)
Chebyshev (tx+agg)	46.06 (0.58)
Chebyshev (tx+agg_red)	45.63 (0.62)
Chebyshev+Lin (tx)	47.62 (0.83)
Chebyshev+Lin (tx+agg)	49.43 (0.62)
Chebyshev+Lin (tx+agg_red)	49.40 (0.79)
Chebyshev+Lin+Skip (tx)	48.14 (0.61)
Chebyshev+Lin+Skip (tx+agg)	50.65 (0.44)
Chebyshev+Lin+Skip (tx+agg_red)	50.54 (0.44)
GAT (tx)	53.46 (0.76)
GAT (tx+agg)	53.73 (0.78)
GAT (tx+agg_red)	53.59 (1.23)
GAT+Lin (tx)	55.84 (0.90)
GAT+Lin (tx+agg)	58.02 (0.36)
GAT+Lin (tx+agg_red)	57.73 (0.41)
GAT+Lin+Skip (tx)	55.93 (0.93)
GAT+Lin+Skip (tx+agg)	59.20 (0.58)
GAT+Lin+Skip (tx+agg_red)	58.77 (0.28)
GATv2 (tx)	56.77 (0.96)
GATv2 (tx+agg)	63.17 (0.51)
GATv2 (tx+agg_red)	62.86 (0.47)
GATv2+Lin (tx)	61.45 (0.75)
GATv2+Lin (tx+agg)	77.51 (0.33)
GATv2+Lin (tx+agg_red)	77.37 (0.29)
GATv2+Lin+Skip (tx)	61.95 (0.93)
GATv2+Lin+Skip (tx+agg)	79.01 (0.41)
GATv2+Lin+Skip (tx+agg_red)	79.23 (0.32)
GCN (tx)	39.28 (0.77)
GCN (tx+agg)	39.49 (0.75)
GCN (tx+agg_red)	39.47 (0.63)
GCN+Lin (tx)	42.74 (0.59)
GCN+Lin (tx+agg)	44.74 (0.36)
GCN+Lin (tx+agg_red)	44.27 (0.44)
GCN+Lin+Skip (tx)	43.01 (0.79)
GCN+Lin+Skip (tx+agg)	45.76 (0.39)
GCN+Lin+Skip (tx+agg_red)	45.68 (0.40)
SAGE (tx)	33.85 (0.34)
SAGE (tx+agg)	35.94 (0.42)
SAGE (tx+agg_red)	35.45 (0.53)
SAGE+Lin (tx)	36.70 (0.51)
SAGE+Lin (tx+agg)	40.24 (0.36)
SAGE+Lin (tx+agg_red)	39.81 (0.44)
SAGE+Lin+Skip (tx)	37.15 (0.57)
SAGE+Lin+Skip (tx+agg)	41.50 (0.32)
SAGE+Lin+Skip (tx+agg_red)	41.03 (0.56)

deviation. The models are evaluated on the basis of the different configurations, as already discussed. As mentioned we trained models for 13000 epochs. Values are related to the training of the models for this amount of epochs. We repeated these tests multiple times, so the reported values are statistics coming from these tests.

The GCN models demonstrate the lowest training times, making them the most efficient in terms of computation. The GATv2 models, on the other hand, have the highest training times, indicating a trade-off between model complexity and training efficiency. The Chebyshev and SAGE models offer a balance between training time and performance, with relatively low standard deviations.

C. DISCUSSION

As a final remark, we emphasize a critical challenge prevalent in digital forensics research, i.e. the opacity of available datasets. The employed dataset lacks a comprehensive documentation and transparency regarding feature selection

and characterization. This opacity substantially impedes our ability to extract meaningful insights into the behavioral patterns of malicious actors engaged in money laundering activities. We reiterate that, at the time of writing this paper, we found no other datasets that were suitably usable in our analysis; thus, we were compelled to exclusively use the dataset under consideration.

Our research demonstrates the potential of transaction graph-based approaches leveraging machine learning mechanisms to identify illicit behaviors. However, our analysis underscores the imperative need for more comprehensive and transparent datasets. Given the practical constraints surrounding the deanonymization of blockchain addresses, a solution might involve the use of synthetic datasets. In this sense, an approach might be based on the use of agent-based modeling. The idea could be to develop synthetic transaction networks using computational agents that simulate complex economic behaviors and interaction patterns typical of cryptocurrency ecosystems. Other approaches might be to use advanced machine learning techniques, such as Generative Adversarial Networks (GANs) or differential privacy techniques [26], to generate synthetic transaction graphs that preserve statistical properties and structural characteristics of real-world cryptocurrency transactions while maintaining anonymity.

By creating controlled, ethically generated environments, researchers can explore complex transaction patterns, test and validate forensic detection algorithms under various simulated scenarios, and thus develop more robust machine learning models for illicit activity detection. Future research should focus on refining these synthetic data generation techniques, ensuring they capture the intricate dynamics of cryptocurrency transaction networks while maintaining the highest standards of methodological rigor and ethical considerations.

While this work focuses on homogeneous graph neural networks, recent developments in heterogeneous GNNs suggest potential further research in this domain [41]. In a parallel research activity, we explored the application of heterogeneous GNNs on a different dataset, where nodes and edges carry additional semantic information [42]. Interestingly, our preliminary findings showed that heterogeneous architectures achieved lower performance compared to that presented in this paper. Further investigation is needed to fully understand the role of heterogeneous information in cryptocurrency transaction networks and to determine whether more sophisticated heterogeneous architectures could leverage this additional information effectively. This remains an open challenge for future research, particularly as new datasets become available and the field continues to evolve.

VI. CONCLUSION

In this paper, we have shown the potential of machine learning methodologies, particularly Graph Neural Networks, in enhancing AML measures within the realm of cryptocurrencies. AML continues to be of utmost importance

in the cryptocurrency sector, as money laundering not only enables and hides criminal activities but also disrupts markets and the wider financial system. By mitigating the effects of such unlawful operations, the research contributes to ongoing initiatives to foster societal welfare. We studied the efficacy of various GNN models, i.e., GCNs, GAT, the Chebyshev convolutional neural networks, and the GraphSAGE networks. As a reference dataset, we used the one provided by Elliptic.

The analysis of the dataset reveals that there is a number of features that are highly correlated. However, the removal of those highly correlated features (i.e., correlation higher than 0.9) seems to provide only marginal improvements. The research outcomes indicated that the Chebyshev convolutional neural networks, and a variant of Graph Attention Network, GATv2, surpassed other models. Another general outcome is that the use of a final linear layer and skip connections further improve the classification performance. At the best of our knowledge, the obtained results improve those available in the existing state of the art.

One aspect that needs further investigation relates to the explainability of GNNs. GNNs are inherently complex models. They involve multiple layers of non-linear transformations and aggregations, which makes it difficult to trace back the influence of individual nodes or edges on the final output. Moreover, the high-dimensional nature of graph data adds further complexity. Each node (transaction) in the graph has numerous features, whose semantics is not revealed in the case of the specific dataset which has been used. This makes it challenging to understand which features or relationships are most influential in the model's decisions.

In fact, GNNs aggregate and propagate information across nodes and their neighbors. While this is key to their performance, it also makes it difficult to determine how information from different parts of the graph contributes to the final output. Thus, while GNNs have shown great promise in various applications, their lack of explainability remains a significant challenge. Addressing this issue is crucial for their adoption in sensitive domains where interpretability is key, such as in AML and forensics in general. Future work in this topic could look into methods for quantifying the importance of individual nodes, edges, or features in the model's decisions.

Finally, while our current study provides valuable insights into transaction graph analysis for detecting potential money laundering activities, it simultaneously highlights the critical need for more transparent, comprehensive, and ethically generated datasets. We claim that synthetic data generation strategies might represent a significant step towards addressing these challenges, offering researchers a powerful toolset for advancing digital forensics methodologies.

REPRODUCIBILITY OF RESULTS

The full source code used for the metrics computation and the reported analysis is available here: <https://github.com/stefano-ferretti/aml-elliptic-gnn/> with an open source license.

REFERENCES

- [1] D. Kogan and D. L. Yermack, "Anti-money laundering compliance for cryptocurrencies," in *Handbook of Digital Finance and Financial Inclusion*. Berlin, Germany: Springer, 2023, pp. 99–123.
- [2] G. Pavlidis, "The birth of the new anti-money laundering authority: Harnessing the power of EU-wide supervision," *J. Financial Crime*, vol. 31, no. 2, pp. 322–330, Mar. 2024.
- [3] N. Pocher, M. Zichichi, F. Merizzi, M. Z. Shafiq, and S. Ferretti, "Detecting anomalous cryptocurrency transactions: An AML/CFT application of machine learning-based forensics," *Electron. Markets*, vol. 33, no. 1, p. 37, Dec. 2023.
- [4] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," in *Proc. KDD Workshop Anomaly Detection Finance*, 2019, pp. 1–7.
- [5] J. Nicholls, A. Kuppa, and N.-A. Le-Khac, "Financial cybercrime: A comprehensive survey of deep learning approaches to tackle the evolving financial crime landscape," *IEEE Access*, vol. 9, pp. 163965–163986, 2021.
- [6] J. Wu, J. Liu, Y. Zhao, and Z. Zheng, "Analysis of cryptocurrency transactions from a network perspective: An overview," *J. Netw. Comput. Appl.*, vol. 190, Sep. 2021, Art. no. 103139. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804521001557>
- [7] S. Marasi and S. Ferretti, "Anti-money laundering in cryptocurrencies through graph neural networks: A comparative study," in *Proc. IEEE 21st Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2024, pp. 272–277.
- [8] M. M. Rathore, S. Chaurasia, and D. Shukla, "Mixers detection in Bitcoin network: A step towards detecting money laundering in cryptocurrencies," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2022, pp. 5775–5782.
- [9] J. Scharfman, "Anti-money laundering compliance for cryptocurrencies," in *Cryptocurrency Compliance and Operations*. Berlin, Germany: Springer, 2022, pp. 91–114.
- [10] V. Chang, P. Baudier, H. Zhang, Q. Xu, J. Zhang, and M. Arami, "How blockchain can impact financial services—The overview, challenges and recommendations from expert interviewees," *Technological Forecasting Social Change*, vol. 158, Sep. 2020, Art. no. 120166.
- [11] M. Jullum, A. Løland, R. B. Huseby, G. Ånonsen, and J. Lorentzen, "Detecting money laundering transactions with machine learning," *J. Money Laundering Control*, vol. 23, no. 1, pp. 173–186, Jan. 2020.
- [12] P. Gerbrands, B. Unger, M. Getzner, and J. Ferwerda, "The effect of anti-money laundering policies: An empirical network analysis," *EPJ Data Sci.*, vol. 11, no. 1, p. 15, Dec. 2022.
- [13] L. Serena, S. Ferretti, and G. D'Angelo, "Cryptocurrencies activity as a complex network: Analysis of transactions graphs," *Peer-Peer Netw. Appl.*, vol. 15, no. 2, pp. 839–853, Mar. 2022.
- [14] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. B. Schardl, and C. E. Leiserson, "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, vol. 34, no. 4, pp. 5363–5370.
- [15] W. W. Lo, G. K. Kulatilleke, M. Sarhan, S. Layeghy, and M. Portmann, "Inspection-L: Self-supervised GNN node embeddings for money laundering detection in Bitcoin," *Appl. Intell.*, vol. 53, no. 16, pp. 19406–19417, Aug. 2023.
- [16] A. O. Adewumi and A. A. Akinyelu, "A survey of machine-learning and nature-inspired based credit card fraud detection techniques," *Int. J. Syst. Assurance Eng. Manage.*, vol. 8, no. 2, pp. 937–953, Nov. 2017.
- [17] R. R. Popat and J. Chaudhary, "A survey on credit card fraud detection using machine learning," in *Proc. 2nd Int. Conf. Trends Electron. Informat. (ICOEI)*, May 2018, pp. 1120–1125.
- [18] J. O. Sinayobye, F. Kiwanuka, and S. K. Kyanda, "A state-of-the-art review of machine learning techniques for fraud detection research," in *Proc. IEEE/ACM Symp. Softw. Eng. Afr. (SEIA)*, May 2018, pp. 11–19.
- [19] I. Mekterović, L. Brkić, and M. Baranović, "A systematic review of data mining approaches to credit card fraud detection," *WSEAS Trans. Bus. Econ.*, vol. 15, pp. 437–444, Jan. 2018.
- [20] I. Sadgali, N. Sael, and F. Benabbou, "Detection of credit card fraud: State of art," *Int. J. Comput. Sci. Netw. Secur.*, vol. 18, no. 11, pp. 76–83, 2018.
- [21] V. Patil and U. K. Lilhore, "A survey on different data mining & machine learning methods for credit card fraud detection," *Int. J. Sci. Res. Comput. Sci., Eng. Inf. Technol.*, vol. 3, no. 5, pp. 320–325, May 2018.

- [22] R. Canillas, R. Talbi, S. Bouchenak, O. Hasan, L. Brunie, and L. Sarra, "Exploratory study of privacy preserving fraud detection," in *Proc. 19th Int. Middleware Conf. Ind.* New York, NY, USA: Association for Computing Machinery, Dec. 2018, pp. 25–31, doi: [10.1145/3284028.3284032](https://doi.org/10.1145/3284028.3284032).
- [23] X. Zhou, Z. Zhang, L. Wang, and P. Wang, "A model based on Siamese neural network for online transaction fraud detection," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Budapest, Hungary, Jul. 2019, pp. 1–7, doi: [10.1109/IJCNN.2019.8852295](https://doi.org/10.1109/IJCNN.2019.8852295).
- [24] Z. Zhang, X. Zhou, X. Zhang, L. Wang, and P. Wang, "A model based on convolutional neural network for online transaction fraud detection," *Secur. Commun. Netw.*, vol. 2018, pp. 1–9, Aug. 2018, doi: [10.1155/2018/5680264](https://doi.org/10.1155/2018/5680264).
- [25] L. Zheng, G. Liu, C. Yan, C. Jiang, M. Zhou, and M. Li, "Improved TrAdaBoost and its application to transaction fraud detection," *IEEE Trans. Computat. Social Syst.*, vol. 7, no. 5, pp. 1304–1316, Oct. 2020.
- [26] Y. Wang, S. Adams, P. Beling, S. Greenspan, S. Rajagopalan, M. Velez-Rojas, S. Mankovski, S. Boker, and D. Brown, "Privacy preserving distributed deep learning and its application in credit card fraud detection," in *Proc. 17th IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun./12th IEEE Int. Conf. Big Data Sci. Eng. (TrustCom/BigDataSE)*, Aug. 2018, pp. 1070–1078.
- [27] G. Tong and J. Shen, "Financial transaction fraud detector based on imbalance learning and graph neural network," *Appl. Soft Comput.*, vol. 149, Dec. 2023, Art. no. 110984. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1568494623010025>
- [28] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.* New York, NY, USA: Association for Computing Machinery, Oct. 2020, pp. 315–324, doi: [10.1145/3340531.3411903](https://doi.org/10.1145/3340531.3411903).
- [29] X. Huang, Y. Yang, Y. Wang, C. Wang, Z. Zhang, J. Xu, L. Chen, and M. Vazirgiannis, "DGraph: A large-scale financial dataset for graph anomaly detection," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst. (NIPS)*, vol. 35, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., Red Hook, NY, USA: Curran Associates, 2022, pp. 22765–22777, Art. no. 1654. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/8f1918f71972789db39ec0d85bb31110-Paper-Datasets_and_Benchmarks.pdf
- [30] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 1–9.
- [31] A. S. Daigavane, B. Ravindran, and G. Aggarwal, "Understanding convolutions on graphs," *Distill*, vol. 6, no. 8, Aug. 2021.
- [32] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, Jan. 2020.
- [33] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [34] W. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 1–11.
- [35] P. Velivckovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–12.
- [36] A. L. Maas, A. Y. Hannun, and A. Y. Ng, "Rectifier nonlinearities improve neural network acoustic models," in *Proc. 30th Int. Conf. Mach. Learn. (ICML)*, 2013, pp. 3–11.
- [37] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *Proc. Int. Conf. Learn. Represent.*, 2022, pp. 1–26.
- [38] M. Newman, *Networks: An Introduction*. Oxford, NY, USA: Oxford Univ. Press, 2010.
- [39] S. Ferretti, "On the modeling of musical solos as complex networks," *Inf. Sci.*, vol. 375, pp. 271–295, Jan. 2017.
- [40] W. L. Hamilton, R. Ying, and J. Leskovec, "Representation learning on graphs: Methods and applications," *IEEE Data Eng. Bull.*, vol. 40, no. 3, pp. 52–74, Sep. 2017.
- [41] T. Liu and H. Meidani, "End-to-end heterogeneous graph neural networks for traffic assignment," *Transp. Res. C, Emerg. Technol.*, vol. 165, Aug. 2024, Art. no. 104695. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0968090X2400216X>
- [42] S. Ferretti, G. D'Angelo, and V. Ghini, "On the use of heterogeneous graph neural networks for detecting malicious activities: A case study with cryptocurrencies," in *Proc. 4th Int. Workshop Open Challenges Online Social Netw.* New York, NY, USA: Association for Computing Machinery, Sep. 2024, pp. 33–40, doi: [10.1145/3677117.3685009](https://doi.org/10.1145/3677117.3685009).



STEFANO FERRETTI (Member, IEEE) is currently a Full Professor with the Department of Computer Science and Engineering, University of Bologna. His current research interests include distributed systems, complex networks, data science, fintech and blockchain technologies, multimedia communications, and hybrid and distributed simulation.



GABRIELE D'ANGELO received the Laurea (summa cum laude) and Ph.D. degrees in computer science from the University of Bologna, Italy, in 2001 and 2005, respectively. He is currently an Assistant Professor with the Department of Computer Science and Engineering, University of Bologna. His research interests include parallel and distributed simulation, distributed systems, and cybersecurity. Since 2011, he has been on the editorial board of *Simulation Modelling Practice and Theory* (SIMPAT) (Elsevier).



VITTORIO GHINI received the Laurea and Ph.D. degrees in computer science from the University of Bologna, in 1997 and 2002, respectively. Since 2011, he has been an Associate Professor with the Computer Science Department, University of Bologna. His research interests include computer networks, mobile computing, containerization, and virtualization technologies.

...